



Restaurant POS

Name: Cantemir Alexandru
Group: 30435

Table of Contents

Deliverable 1.....	3
Project Specification	3
Functional Requirements	3
Use Case Model	3
Use Cases Identification:.....	3
UML Use Case Diagrams.....	5
Supplementary Specification	5
Non-functional Requirements	5
Design Constraints	6
Glossary.....	Error! Bookmark not defined.
Deliverable 2.....	7
Domain Model	7
Architectural Design	8
Conceptual Architecture	8
Package Design	9
Component and Deployment Diagram	9
Deliverable 3.....	11
Design Model.....	11
Dynamic Behavior	11
Class Diagram	12
Data Model.....	13
System Testing.....	14
Future Improvements.....	14
Conclusion.....	15
Bibliography	15

Deliverable 1

Project Specification

The Restaurant Point of Sale (POS) and Management System is a full-stack web application designed to handle daily restaurant operations. It provides a unified platform for real-time ordering, employee time tracking and administrative oversight.

Functional Requirements

- **Real-time ordering:** After a waiter sends an order through the application, the kitchen interface updates in real-time with the necessary instructions.
- **Rapid authentication:** A PIN-pad is used to authenticate users with a 4 digit PIN.
- **Role Based Dashboards:** Based on the user's role, a specific dashboard is loaded that matches the needs of the user.
- **Attendance and Salary tracking:** An automated ledger that keeps track of shift start and end times along with the hourly rate.
- **CRUD operations:** A manager can see, create, update and delete users, roles, products and item types.
- **Table and Ticket tracking:** Each table can have a name/number to better identify the orders and each item inside an order can have a description (e.g. "extra crispy").

Use Case Model

Use Cases Identification:

Use-Case: Place New Order

Level: Low

Primary Actor: Waiter

Main success scenario:

1. Waiter successfully authenticates using the 4-digit PIN-pad.
2. Waiter selects an open table (e.g., Table 4) from the dining room dashboard.
3. Waiter adds specific products to the order and includes custom text descriptions if necessary (e.g., "extra crispy").
4. Waiter submits the order.
5. The system saves the order, sets the status to "In Progress," and instantly pushes the ticket data to the kitchen interface.

Extensions:

1. Invalid PIN entered: The system denies access, flashes an error, and clears the keypad for a retry.
2. Waiter cancels the order before submission: The system drops the temporary items and returns the waiter to the main dashboard without saving to the database.

Use-Case: Shift Clock-In

Level: Low

Primary Actor: Waiter

Main success scenario:

1. Employee enters their PIN to access their specific role-based dashboard.
2. Employee selects the "Clock In" action.
3. The system queries the database to retrieve the employee's current hourly rate.
4. The system creates a new shift record, capturing the exact current timestamp and locking in the hourly wage.
5. The UI confirms a successful clock-in and updates the employee's status to active.

Extensions:

1. Employee is already currently clocked in: The system hides the "Clock In" button and instead presents the "Clock Out" workflow to prevent duplicate shifts.
2. Database fails to retrieve wage data: The system aborts the clock-in process and alerts the user to contact a manager, preventing a shift from starting with a zeroed-out wage.

Use-Case: Securely Archive a Departing Employee

Level: High

Primary Actor: Manager

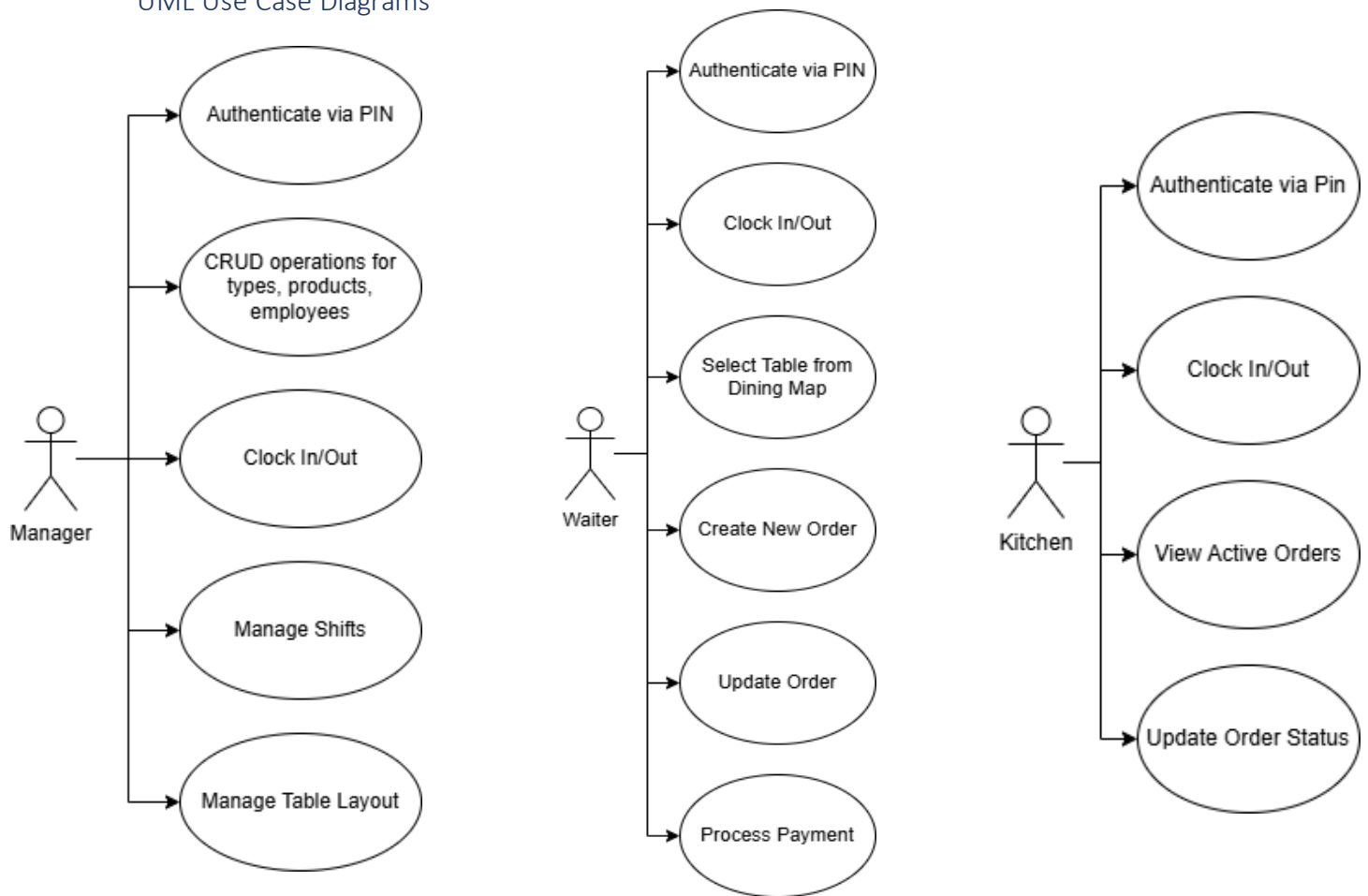
Main success scenario:

1. Manager successfully logs in and navigates to the administrative User Management dashboard.
2. Manager selects an active employee from the system roster and initiates the "Delete" action.
3. The system confirms the destructive action via a popup prompt.
4. Upon confirmation, the backend executes a soft-delete by stamping the deleted_at column with the current exact timestamp.
5. The system removes the user from the active login pool while permanently preserving all of their historical orders and shift data.

Extensions:

1. Manager attempts to archive their own admin account: The system blocks the request to prevent a total logout scenario.
2. The selected employee currently has an open, active shift: The system interrupts the archive process and prompts the manager to manually clock the employee out first.

UML Use Case Diagrams



Supplementary Specification

Non-functional Requirements

1. Performance and Responsiveness

The system's frontend interface and backend API must process standard operations (authenticating, opening a table, adding an item) with extremely low latency, returning visual confirmation as fast as possible.

In a high-volume restaurant, waitstaff cannot be bottlenecked by loading screens during a dinner rush. Utilizing FastAPI's asynchronous architecture and React's immediate state updates ensures the system feels instantaneous and does not slow down floor operations.

2. Data Integrity and Auditability

The database must enforce strict relational constraints and maintain an immutable historical record of all financial and attendance data, preventing accidental data loss or cascading historical errors.

Restaurant accounting requires absolute precision. By implementing a `deleted_at` soft-delete for user accounts and capturing a static snapshot of an employee's `hourly_rate` at the exact time of clock-in, the system guarantees that past payroll and order histories remain perfectly intact, even if a manager deletes an employee or changes a menu price later.

3. Security and Access Control

The application must enforce strict Role-Based Access Control across all frontend views and backend API routes to prevent unauthorized data manipulation.

A waiter must be able to open a table, but they should never have the ability to view payroll data or change a product's price. The implementation utilizes secure session handling and role ID checks to ensure that the administrative dashboard and destructive CRUD operations are strictly isolated to Manager-level accounts.

4. Usability and Efficiency

The frontend Point of Sale terminal must be highly intuitive, optimized for rapid touchscreen interactions, and require minimal training for new staff to operate.

The hospitality industry frequently deals with high staff turnover. By utilizing a straightforward numerical PIN-pad for rapid authentication and a clear, color-coded visual grid for table zones, the UI reduces cognitive load, minimizes human input error, and allows new hires to start taking orders on their first day.

Design Constraints

1. Mandated Languages and Frameworks

The system is strictly constrained to a decoupled JavaScript and Python environment to ensure a clear separation of concerns between the user interface and the core business logic:

Frontend: React.js (initialized via Vite) is mandated for building a dynamic, component-based Single Page Application (SPA).

Styling: Tailwind CSS is mandated for utility-first, responsive UI design to maintain visual consistency without relying on fragmented custom CSS stylesheets.

Backend: Python 3.x with the FastAPI framework is mandated to provide high-performance, asynchronous RESTful API endpoints and strict data validation via Pydantic.

2. Architectural Paradigm

The application must adhere strictly to a standard Client-Server architecture: The frontend React client and the backend FastAPI server must remain completely decoupled and operate on separate runtime environments. All communication between the client and server is restricted to standard HTTP/HTTPS requests exchanging JSON payloads.

3. Database and Data Integrity Constraints

Due to the financial and time-tracking nature of a Point of Sale system, the data layer is constrained to a strictly relational model:

Database Engine: A robust SQL database (like PostgreSQL) is mandated to ensure ACID compliance, strict Foreign Key enforcement, and exact data typing (e.g., using NUMERIC for hourly rates and prices to avoid floating-point math errors).

ORM Requirement: All backend database interactions must be abstracted through the SQLAlchemy Object-Relational Mapper to prevent raw SQL injection vulnerabilities and maintain maintainable code.

4. Prescribed Development Tools

To maintain consistency across the development and testing environments, specific tools are mandated:

Database Management: JetBrains DataGrip is used for direct schema inspection, manual table creation, and raw query testing.

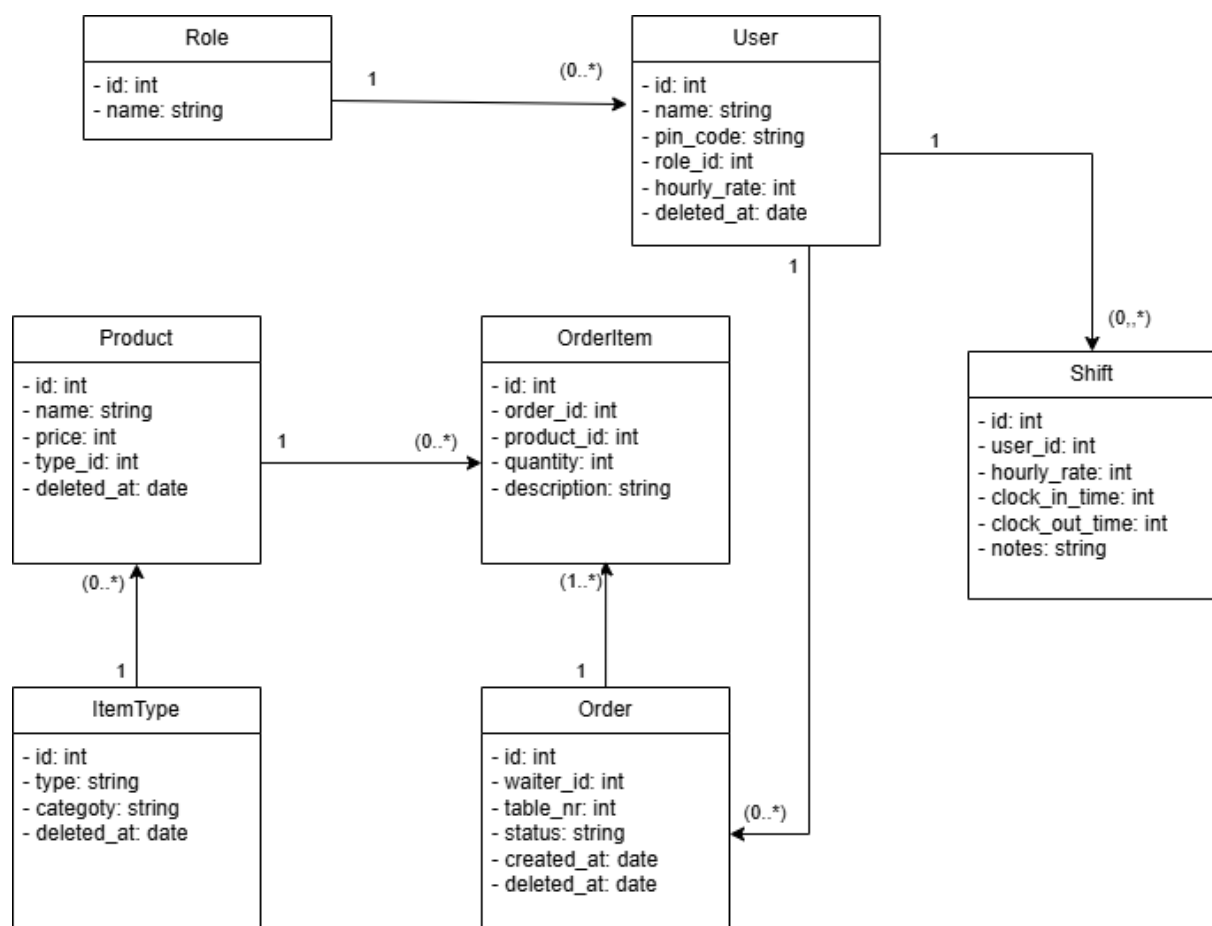
Package Management: pip must be used for Python backend dependencies, and npm must be used for frontend Node.js modules.

Security: Based on project scope and UI speed requirements, standard text validation is used for PIN authentication rather than cryptographic hashing, constraining the system to physically secure, internal restaurant networks.

Deliverable 2

Domain Model

At the center of the system is the User (representing staff members), whose system permissions are defined by a singular Role (e.g., Manager or Waiter). The model captures workforce management by linking Users to multiple Shifts, which record precise time tracking and hourly wages for payroll. On the service side, the menu is structured hierarchically, where broad ItemTypes act as umbrellas for multiple specific Products (menu items), allowing for intuitive navigation. The operational bridge between the staff and the menu is the Order entity, which represents an active table tab. Every Order is tethered to the specific User (Waiter) who opened it, ensuring accountability and workflow tracking from the moment a table is seated to the final checkout.



Architectural Design

Conceptual Architecture

The Restaurant POS system is designed using a Client-Server Architecture that implements a Three-Tier structure. Communication between the client and the server adheres to the REST style, ensuring stateless, predictable, and standardized interactions via HTTP protocols.

The Three-Tier Structure:

1. **Presentation Tier (Client/Frontend):** This tier is responsible for all user interfaces, including the Waiter Dashboard (table management, order entry) and the Manager Dashboard (employee management, shift tracking). It operates entirely in the user's web browser or on a restaurant tablet.
2. **Application Logic Tier (Server/Backend):** This tier handles all business rules, authentication logic, shift calculations, and data processing. It acts as the intermediary, receiving RESTful API requests from the React frontend, processing them, and interacting with the database.
3. **Data Tier :** This tier ensures data persistence, structural integrity, and relational mapping (e.g., ensuring orders are strictly tied to specific users and products).

Architectural Pattern:

Internally, the backend application is structured using an adaptation of the classic MVC pattern, optimized for modern API development:

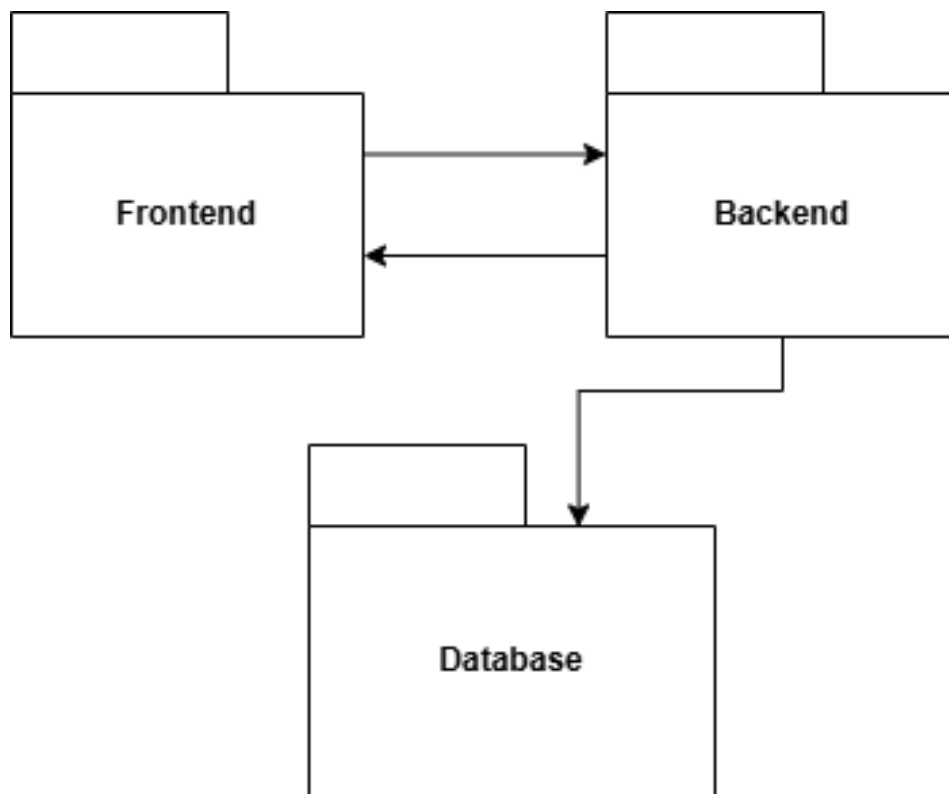
- **Model (Data Representation):** Implemented using SQLAlchemy ORM (Object-Relational Mapping). The models (`models.py`) define the exact database schema and handle the direct reading and writing of data rows (e.g., User, Shift, Order).
- **View (Data Serialization/Validation):** Implemented using Pydantic schemas (`schemas.py`). Instead of rendering HTML pages, the "Views" in this REST API define the exact JSON structure sent to and received from the React client, ensuring strict data validation before it ever touches the database.
- **Controller (Routing Logic):** Implemented using FastAPI routers (`main.py`). The controllers define the API endpoints, receive the client's HTTP requests, invoke the necessary database Models, and return the formatted Views.

Motivation and Justification:

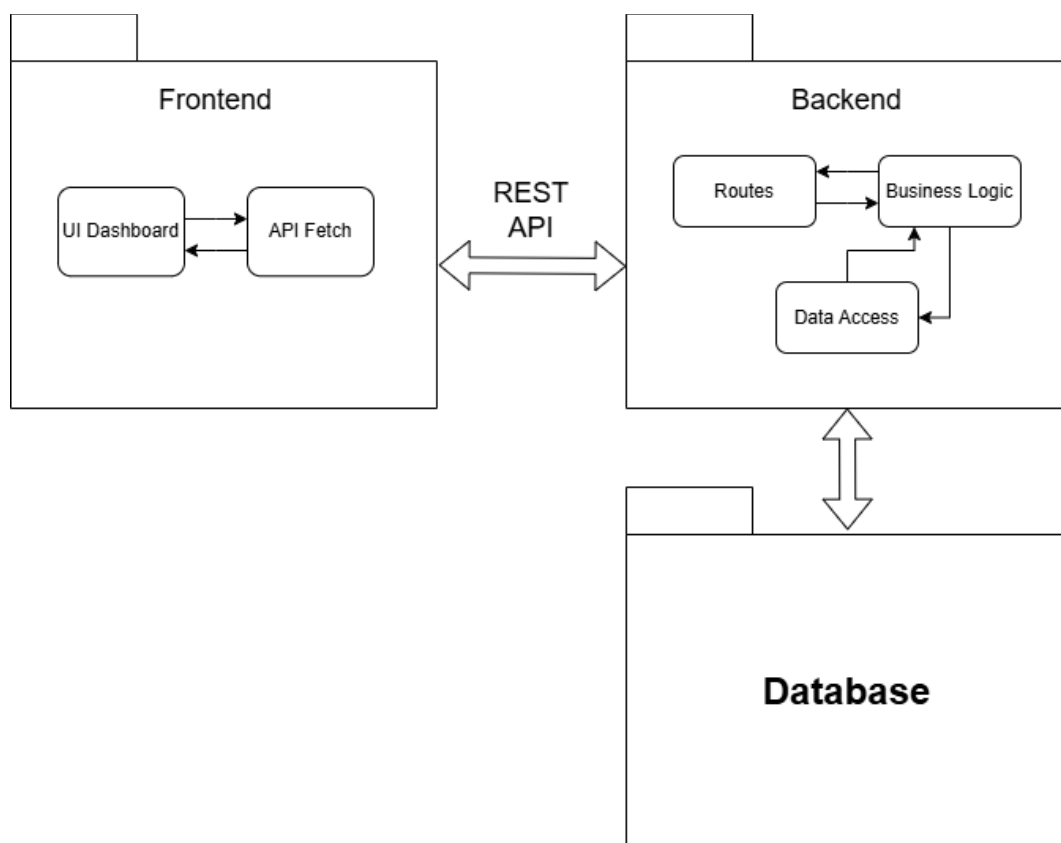
This architectural style was chosen for several critical reasons:

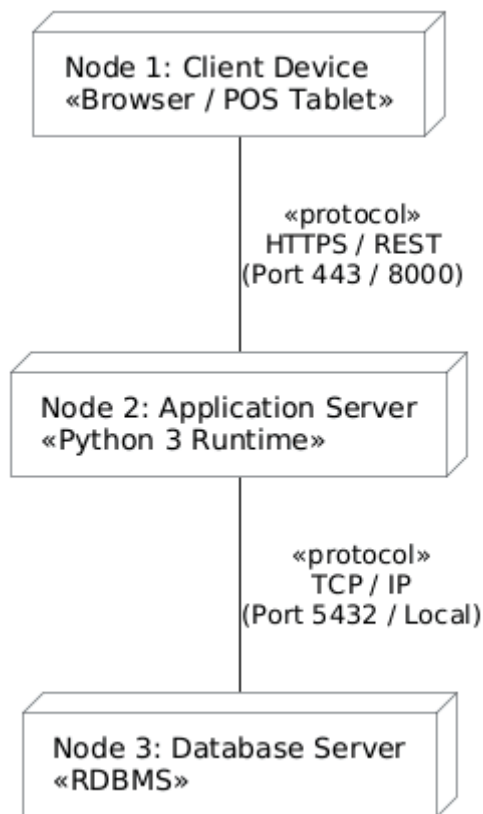
- **Separation of Concerns:** By decoupling the frontend from the backend, the user interface can be completely redesigned or updated without risking damage to the core business logic or database structure.
- **Platform Independence:** A RESTful API allows the system to be highly extensible. The restaurant can deploy the current React web app on POS terminals, and in the future, seamlessly build a native iOS/Android app for waiters using the exact same backend.
- **Performance and Asynchrony:** FastAPI is natively asynchronous, meaning the server can handle multiple waiters submitting orders simultaneously during peak restaurant hours without bottlenecks or crashing.

Package Design



Component and Deployment Diagram

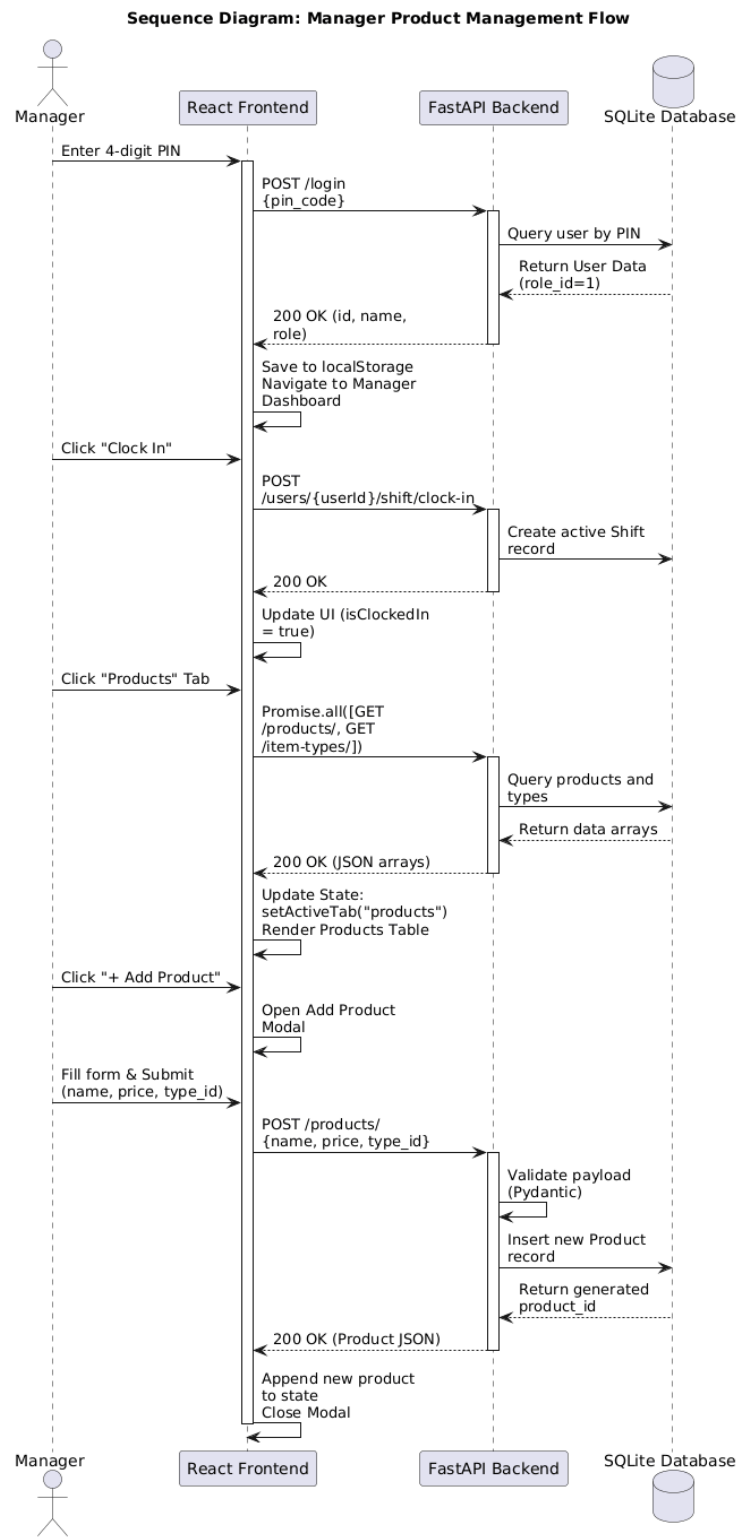
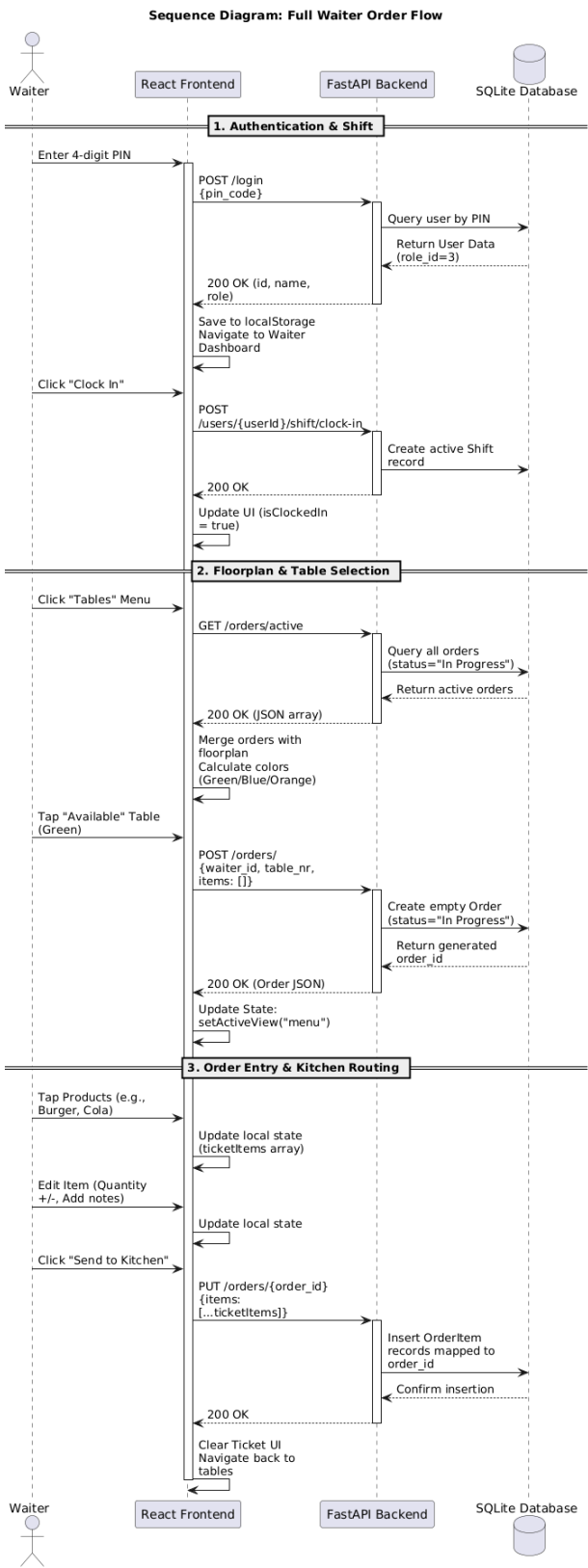




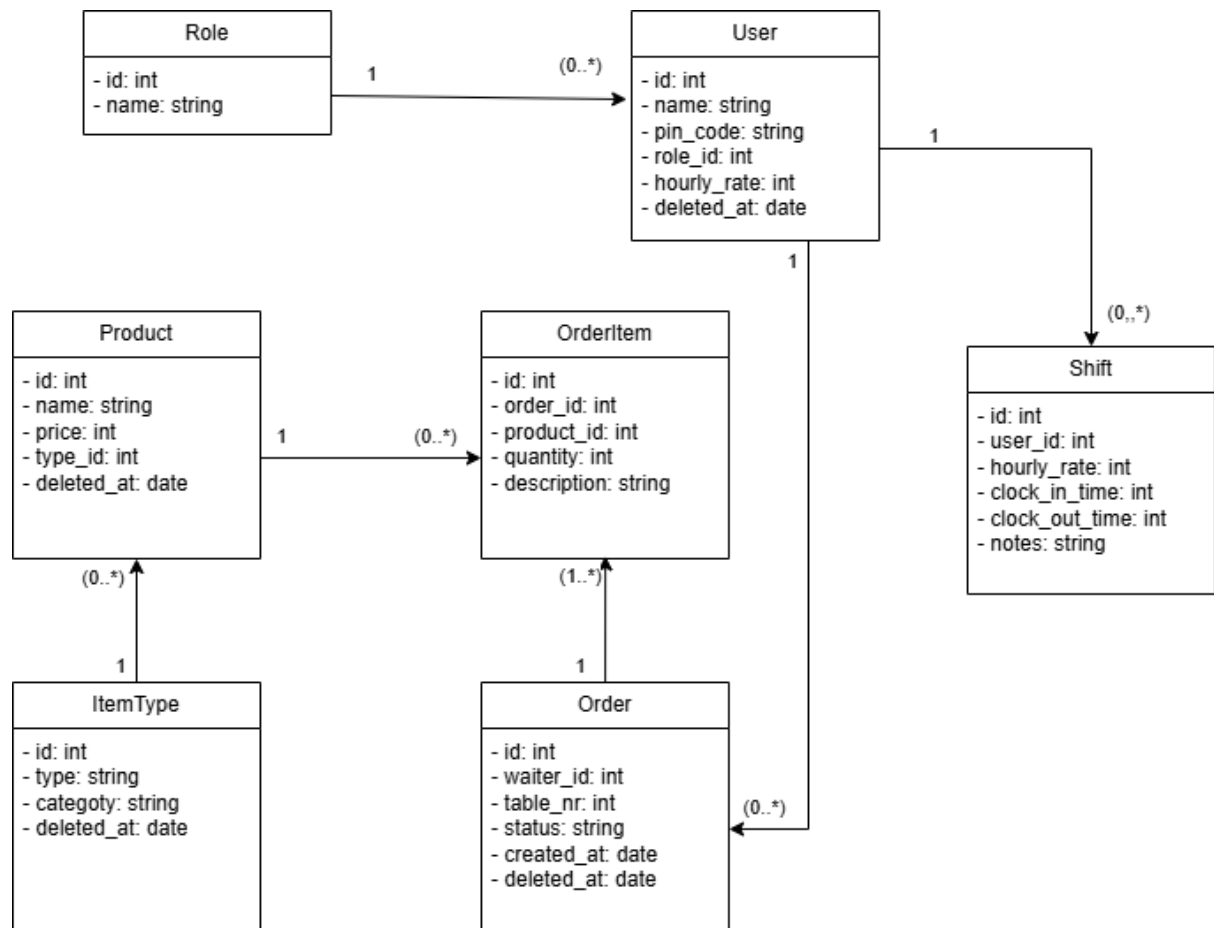
Deliverable 3

Design Model

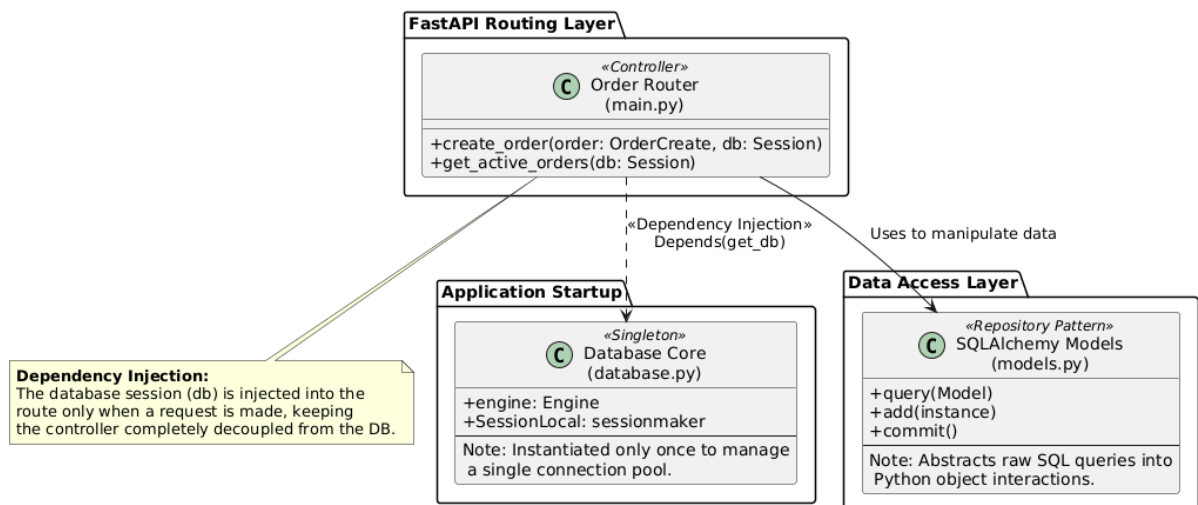
Dynamic Behavior



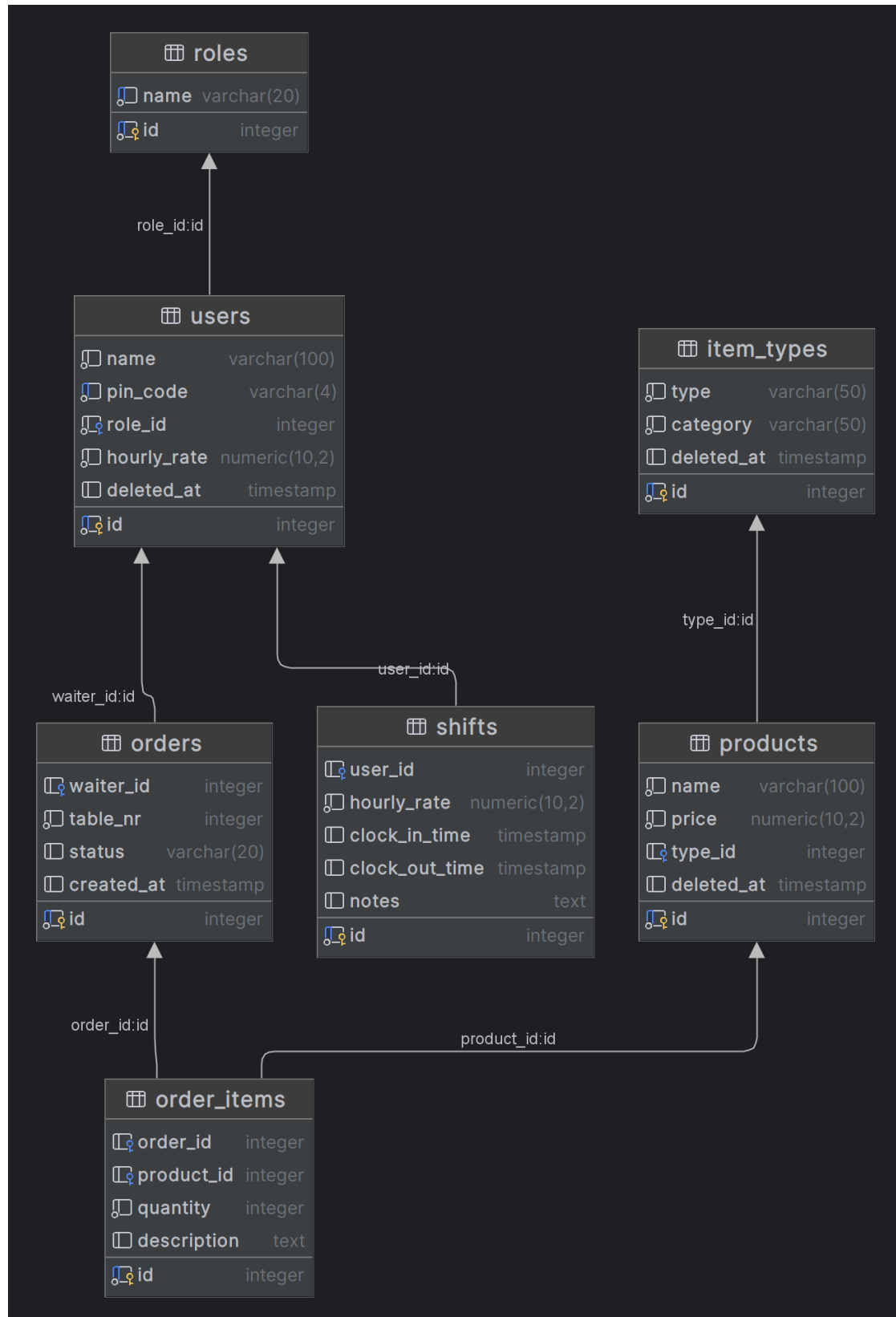
Class Diagram



Architectural Design Patterns



Data Model



System Testing

1. Testing Methods:

- **Unit Testing:** This method tests individual, isolated components of the software without relying on external systems like the database or the network.
 - *Backend Application:* We use the pytest framework to test the FastAPI Pydantic schemas, ensuring that data validation rules (like preventing a negative price for a product) work perfectly before hitting the database.
 - *Frontend Application:* We test React helper functions and local state calculations (such as the cart total calculation in the POS menu) to ensure mathematical accuracy.
- **Integration Testing:** This method tests how different modules of the application work together. We use FastAPI's TestClient paired with an in-memory SQLite database to safely simulate API requests. This ensures that when the React frontend sends an HTTP request, the FastAPI router correctly processes it, the SQLAlchemy repository writes it, and the database successfully stores it.

2. Test Cases:

Test Case 1: Product Validation (Backend Unit Test)

- **Objective:** Ensure the system rejects invalid menu items at the model level.
- **Precondition:** The system attempts to instantiate a ProductCreate Pydantic schema.
- **Action:** Provide the schema with a valid name but a negative price (-5.00).
- **Expected Result:** The Pydantic model throws a ValidationError due to the strict numerical constraints on the price field, preventing bad data from reaching the database.

Test Case 2: Ticket Total Calculation (Frontend Unit Test)

- **Objective:** Verify the mathematical accuracy of the active order ticket on the Waiter Dashboard.
- **Precondition:** The core calculateTicketTotal logic is extracted into a pure, testable function.
- **Action:** Pass an array of mock items (e.g., 2 items at \$2.50, 1 item at \$12.00) into the function.
- **Expected Result:** The function accurately multiplies quantities by price and strictly returns the float value 17.00.

Test Case 3: Claiming an Available Table (Backend Integration Test)

- **Objective:** Verify the end-to-end flow of opening a new restaurant table ticket.
- **Precondition:** A test database is seeded with a valid mock Waiter.
- **Action:** The API client sends a POST /orders/ payload containing the specific waiter_id and table_nr: 4.
- **Expected Result:** The FastAPI backend validates the payload, creates an Order record with the status "In Progress", and returns an HTTP 200 OK with the complete generated JSON object.

Future Improvements

1. Dynamic Visual Floorplan Builder:

Currently, the physical layout of the restaurant is represented via a static grid on the frontend. A future iteration would introduce a drag-and-drop canvas interface within the Manager Dashboard. This would allow administrators to visually draw tables, map them to

dynamic RestaurantTable database records, and instantly update the spatial layout on the waiter terminals.

2. Automated Inventory Tracking & Recipe Mapping:

To extend the system's operational value, an Ingredients table could be introduced and relationally mapped to the Products table. This would allow the backend to automatically deduct ingredient quantities from the inventory every time an order is completed, triggering automated "Low Stock" alerts to the Manager Dashboard when thresholds are breached.

3. Third-Party Payment Gateway Integration:

Connecting external financial APIs would allow the system to process credit card transactions directly on the Waiter Dashboard. The backend would securely handle webhook confirmations, automatically linking payment confirmation IDs to the orders table and shifting the order status to "Paid and Closed" without manual intervention.

Conclusion

This project successfully delivered a fast, reliable, and scalable Point of Sale system designed for modern restaurant management. By combining a responsive React frontend with a high-speed FastAPI backend, the application provides a seamless and intuitive experience for the staff. Key features, such as real-time WebSocket synchronization, ensure instant communication between the waiters and the kitchen, while the strict database design keeps all orders, inventory, and shift data completely secure. Ultimately, the system is fully tested, easy to maintain, and provides a strong technological foundation that is ready to handle the daily demands of a busy restaurant environment.

Bibliography

<https://tailwindcss.com/docs/styling-with-utility-classes>

<https://fastapi.tiangolo.com/>